Taller PokéDex Explorer — Guía del estudiante

Oct 20, 2026 · @Paula Andrea

Información general

Cada grupo construirá **PokéDex Explorer**, una app con Expo que consume la PokéAPI, lista Pokémon, permite buscarlos por nombre y muestra su ficha con imagen, tipos y estadísticas.

Aspecto	Detalle
Duración	4 horas (2 sesiones de 2 h)
Modalidad	Parejas o grupos de 3
Tecnologías	React Native, Expo, JavaScript, `fetch`
Conceptos previos	Componentes core (View, Text, TextInput, Button, Image), StyleSheet, SafeAreaView, useState, useEffect
Entrega	Repositorio o ZIP del proyecto + capturas + respuestas de reflexión

Objetivos de aprendizaje. Al terminar, el estudiante podrá:

1. Consumir una API REST con `fetch` y `async/await`, manejando respuesta, error y carga.
2. Diseñar una arquitectura propia que separe la lógica de red de la interfaz, y justificarla.
3. Controlar cuándo se ejecuta una petición con el array de dependencias de `useEffect`.
4. Renderizar listas eficientes con `FlatList` y mostrar imágenes remotas con `Image`.
5. Diseñar estados de interfaz: cargando, error, vacío y éxito.

Reglas del código. Código en inglés, textos de la interfaz en español. Cada concepto visto en clase se marca con un comentario

La API: PokéAPI

La PokéAPI es pública, gratuita y no requiere API key. Todas las rutas parten de `https://pokeapi.co/api/v2/` y responden JSON.

Endpoint	Uso en la app	Ejemplo
`GET /pokemon?limit={n}&offset={m}`	Lista paginada	`/pokemon?limit=20&offset=0`
`GET /pokemon/{nombre o id}`	Ficha de un Pokémon	`/pokemon/pikachu` o `/pokemon/25`

Respuesta de la lista. Solo trae nombre y URL; el id se extrae del final de la URL.

```
{
  "count": 1302,
  "next": "https://pokeapi.co/api/v2/pokemon?offset=20&limit=20",
  "previous": null,
  "results": [
    { "name": "bulbasaur", "url": "https://pokeapi.co/api/v2/pokemon/1/" }
  ]
}
```

Campos de la ficha que usaremos (la respuesta real es mucho más grande):

```
{
  "id": 25,
  "name": "pikachu",
  "height": 4,
  "weight": 60,
  "types": [{ "slot": 1, "type": { "name": "electric" } }],
  "stats": [{ "base_stat": 35, "stat": { "name": "hp" } }],
  "sprites": {
    "other": { "official-artwork": { "front_default": "https://...25.png" } }
  }
}
```

- `height` viene en decímetros y `weight` en hectogramos: divide entre 10 para mostrar metros y kilogramos.
- Si el nombre no existe, la API responde **404** con texto plano, no JSON. Hay que revisar `response.ok` antes de llamar `response.json()`.

- Los nombres deben ir en minúsculas y sin espacios al final.
- Para la imagen de la lista, sin pedir la ficha completa: `https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/official-artwork/{id}.png`.

Actividad previa (10 min). Abre ambos endpoints en el navegador o en Postman e identifica dónde están el id, los tipos y la imagen.

Preparación del entorno

El proyecto usa la plantilla `blank` de Expo para empezar con un solo `App.js`, sin Expo Router.

```
npx create-expo-app@latest pokedex-explorer --template blank
cd pokedex-explorer
npx expo install react-native-safe-area-context
npx expo start
```

Escanea el QR con Expo Go o presiona `i` (simulador iOS) o `a` (emulador Android). No se necesita instalar ninguna librería HTTP: `fetch` viene incluido.

Arquitectura: decisión del equipo. No hay una estructura de carpetas obligatoria. Cada equipo decide cómo organizar sus archivos, componentes y la lógica que consume la API. La única regla: las llamadas a la API no deben quedar mezcladas con los estilos y el JSX de cada componente.

Documenten en el README el árbol de carpetas y la responsabilidad de cada archivo. En la sustentación cualquier integrante deberá explicar por qué lo organizaron así.

Qué debe construir el equipo

La app tiene una pantalla principal con dos vistas: la lista de Pokémon y la ficha de detalle. Cómo las implementan es decisión del equipo; lo que sigue describe el comportamiento esperado, no el código.

Vista de lista

1. Encabezado con el título "PokéDex Explorer".

2. Barra de búsqueda con un `TextInput` (placeholder "Busca por nombre o número…") y un botón "Buscar". La búsqueda acepta nombre o número e ignora mayúsculas y espacios al inicio o al final.
3. Cuadrícula de 2 columnas con tarjetas: imagen, número con 3 dígitos (`#025`) y nombre con la primera letra en mayúscula.
4. Al abrir la app se cargan 20 Pokémon; al llegar al final de la lista se cargan 20 más (scroll infinito).
5. Mientras se cargan más, se ve un indicador al final de la lista.
6. Tocar una tarjeta abre la ficha de ese Pokémon.

Vista de detalle

1. Imagen grande, nombre y número.
2. Un distintivo por cada tipo (fuego, agua, planta…) con un color propio por tipo; la paleta la elige el equipo.
3. Altura en metros y peso en kilogramos.
4. Las 6 estadísticas base, cada una con su valor y una barra de ancho proporcional (el máximo posible es 255).
5. Botón "Volver a la lista".

Estados de la interfaz

Toda petición a la API debe mostrar al usuario en qué estado está:

Estado	Qué ve el usuario
Cargando	Indicador de carga con texto
Error	Mensaje claro en español y botón "Reintentar"
Sin resultados	Mensaje cuando el nombre buscado no existe
Éxito	La lista o la ficha

Pistas técnicas

- Una `Image` con URL remota necesita `width` y `height` explícitos o no se ve.
- Antes de llamar `response.json()` revisa `response.ok` : un nombre inexistente responde 404 con texto, no JSON.

- Para listas largas usa `FlatList` (`numColumns`, `keyExtractor`, `onEndReached`), no `ScrollView` con `map`.
- Número con ceros a la izquierda: `String(id).padStart(3, '0')`.
- En el `TextInput` de búsqueda usa `autoCapitalize="none"` y `autoCorrect={false}`.
- Revisa con cuidado el array de dependencias de cada `useEffect` para que la petición no se repita sin parar.